

UNIVERSIDADE DE BRASÍLIA – UNB
Departamento de Ciência da Computação – CIC
Disciplina: Redes de Computadores

Gabriel Gonçalves Caldo – 231034627
Mario Augusto Vieira dos Santos – 231035778
Daniel Rodrigues de Abreu – 241038540

Grupo 2

RELATÓRIO TÉCNICO

Projeto de Chat P2P

Brasília, DF
2026

Sumário

1	Introdução	2
2	Arquitetura da Solução	2
3	Protocolos Implementados com Exemplos de Funcionamento	4
3.1	Estabelecimento e Sessão Inicial (HELLO / HELLO_OK)	4
3.2	Keep-Alive (PING / PONG)	5
3.3	Mensagens de Chat e Encerramento	5
4	Interface de Usuário (CLI)	7
5	Instruções Detalhadas para Execução	7
5.1	Requisitos Mínimos para Executar o Programa em Linux	7
5.2	Compilação do Código	8
5.3	Bibliotecas Necessárias e Como Instaladas	8
5.4	Passos Necessários para Executar o Programa	8
6	Apêndice: Códigos-Fonte	10
6.1	main.py	10
6.2	config.py	16
6.3	config.json	19
6.4	logging_config.py	20
6.5	cli.py	23
6.6	message_router.py	31
6.7	peer_server.py	36
6.8	peer_connection.py	42
6.9	peer_table.py	54
6.10	rendezvous_connection.py	64
6.11	reconnection_manager.py	71

1 Introdução

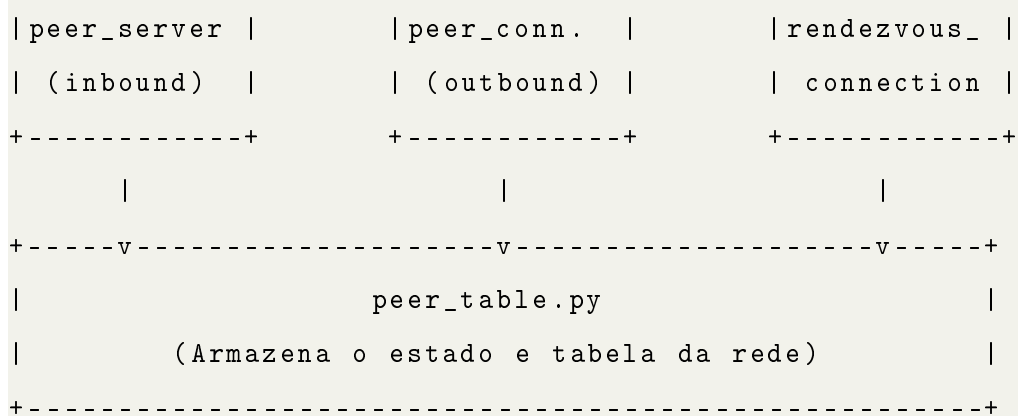
Este relatório técnico descreve a implementação do cliente de Chat P2P desenvolvido pelo Grupo 2 para a disciplina de Redes de Computadores da Universidade de Brasília (UnB). O sistema é capaz de se registrar e buscar nós em um servidor Rendezvous central, além de estabelecer conexões diretas TCP (*Peer-to-Peer*) persistentes com outros clientes para a troca de mensagens, seguindo os requisitos estipulados na especificação do projeto. A comunicação ocorre 100% em formato P2P após a descoberta de nós, isto é, sem saltos intermediários e com a ausência de um ponto centralizado de troca de mensagens.

2 Arquitetura da Solução

A arquitetura do software foi concebida aproveitando ao máximo a biblioteca nativa assíncrona do Python (`asyncio`). Isto permite o tratamento concorrente de múltiplas conexões de rede numa *thread* única, conferindo eficiência e simplicidade no controle de estado.

Abaixo, apresentamos o diagrama estrutural de como os módulos principais se organizam e interagem de forma descentralizada:





Nossa implementação divide a lógica em diversos módulos desacoplados:

- **main.py**: Arquivo principal. Carrega as definições, registra no servidor central via `rendezvous_connection.py` e inicia o ciclo do *event loop* do `asyncio`, englobando interface com usuário e servidor.
- **config.py**: Classe central para carregar metadados e configurações do usuário a partir de `config.json` (como TTL e features do protocolo).
- **logging_config.py**: Módulo contendo lógicas avançadas (como `ReadlineConsoleHandler`) para harmonizar as saídas assíncronas do terminal sem quebrar a renderização da interface do usuário.
- **peer_server.py**: Define o servidor local para escuta de conexões *inbound* (recebidas) de outros nós P2P.
- **peer_connection.py**: Implementa o manipulador de cada conexão TCP bidirecional (*inbound* ou *outbound*). Ele abstrai a leitura delimitada por `\n`, lida com pacotes *keep-alive* (PING/PONG), negocia a sessão inicial com HELLO/HELLO_OK e despacha as mensagens de chat recebidas.
- **rendezvous_connection.py**: Concentra a requisição sobre *sockets* para interagir com o servidor de Rendezvous, tratando comandos de REGISTER, DISCOVER e UNREGISTER.
- **message_router.py**: O *Router* é responsável por processar requisições P2P unificadas, evitando a duplicação de lógicas de protocolo para *inbound* e *outbound*. Se for uma mensagem *unicast* (tipo SEND), ele exhibe e envia o ACK. Se for *broadcast* (tipo PUB), exhibe publicamente e também auxilia no *keep-alive* gerenciando PING/PONG.

- `peer_table.py`: Funciona como a tabela de vizinhos ou estado (*State*). Ele armazena metadados de outros nós registrados na mesma sala (*namespace*) e controla a saúde dos enlaces, marcando nós obsoletos como **STALE**.
- `reconnection_manager.py`: Cuida das tarefas em plano de fundo de reconexões usando políticas configuráveis, como o *backoff exponencial*, definindo quantas vezes uma conexão rompida deve ser retentada.
- `cli.py`: Fornece a interface visual baseada em linha de comando interativa para acesso humano. Transmite chamadas dos comandos (ex: `/msg`, `/pub`, `/conn`) para as outras camadas.

O cliente lê dados seriais formatados estritamente em JSON de UTF-8. Assim, a comunicação possui tamanho restrito e delimitador forte para a quebra de 32KB em buffer TCP, conforme exigido.

3 Protocolos Implementados com Exemplos de Funcionamento

O projeto concretiza diferentes interações ao nível de aplicação na rede, conforme o seguinte mapeamento. Todos os comandos JSON terminam em um caractere de nova linha (`\n`).

3.1 Estabelecimento e Sessão Inicial (HELLO / HELLO_OK)

Todo enlace começa com o reconhecimento de identidade do *peer*.

- **Cenário:** Alice conecta-se fisicamente no IP do Bob e emite um HELLO. Bob responde informando que aceita o fluxo de dados através do HELLO_OK.
- **Exemplo Requisição (Alice → Bob):**

```
{
  "type": "HELLO",
  "peer_id": "alice@UnB",
  "version": "1.0",
  "features": ["ack", "metrics"],
```

```
    "ttl": 1
}
```

- **Exemplo Resposta (Bob → Alice):**

```
{
  "type": "HELLO_OK",
  "peer_id": "bob@UnB",
  "version": "1.0",
  "features": ["ack", "metrics"],
  "ttl": 1
}
```

3.2 Keep-Alive (PING / PONG)

Para mensurar RTT (Tempo de Ida e Volta) e evidenciar que a conexão TCP não foi morta por *middleboxes* como NATs e *firewalls*, enviamos ping periódico:

- **Exemplo PING:**

```
{
  "type": "PING",
  "msg_id": "uuid",
  "timestamp": "2025-10-27T10:00:00Z",
  "ttl": 1
}
```

- **Exemplo PONG:**

```
{
  "type": "PONG",
  "msg_id": "uuid",
  "timestamp": "2025-10-27T10:00:00Z",
  "ttl": 1
}
```

3.3 Mensagens de Chat e Encerramento

As mensagens usam a marcação **SEND** (Unicast), **ACK** (Recibo de Recebimento) e **PUB** (Broadcast). Ao fim de tudo, **BYE** e **BYE_OK** gerenciam o fechamento.

- Exemplo de SEND (Unicast com exigência de recibo):

```
{
  "type": "SEND",
  "msg_id": "uuid",
  "src": "alice@UnB",
  "dst": "bob@UnB",
  "payload": "Olá!",
  "require_ack": true,
  "ttl": 1
}
```

- Exemplo de ACK correspondente:

```
{
  "type": "ACK",
  "msg_id": "uuid",
  "timestamp": "2025-10-27T10:00:01Z",
  "ttl": 1
}
```

- Exemplo de PUB (Difusão Global e Namespace):

```
{
  "type": "PUB",
  "msg_id": "uuid",
  "src": "alice@UnB",
  "dst": "*",
  "payload": "Mensagem global",
  "require_ack": false,
  "ttl": 1
}
```

- Exemplo de BYE (Encerramento do enlace TCP):

```
{
  "type": "BYE",
  "msg_id": "uuid",
  "src": "alice@UnB",
  "dst": "bob@UnB",
  "reason": "Encerrando sessão",
  "ttl": 1
}
```

```
}
```

- **Exemplo de BYE_OK (Confirmação de encerramento):**

```
{
  "type": "BYE_OK",
  "msg_id": "uuid",
  "src": "bob@UnB",
  "dst": "alice@UnB",
  "ttl": 1
}
```

4 Interface de Usuário (CLI)

A aplicação fornece uma interface interativa de linha de comando para interação humana em tempo real com a rede. Os comandos suportados e suas respectivas funções são descritos abaixo:

Comando	Função
/peers [* #namespace]	Descobrir e listar peers ativos na tabela
/msg <peer_id> <mensagem>	Enviar mensagem direta (Unicast)
/pub * <mensagem>	Enviar broadcast global
/pub #<namespace> <mensagem>	Enviar mensagem para todos do namespace
/conn	Mostrar conexões ativas (inbound/outbound)
/rtt	Exibir RTT médio por peer via keep-alive
/reconnect	Forçar reconciliação de peers (desconectados)
/log <Nível>	Ajustar nível de log (DEBUG, INFO...)
/quit	Encerrar aplicação de forma segura e limpa

5 Instruções Detalhadas para Execução

Esta subseção responde às perguntas contidas na documentação técnica para viabilizar a implantação local pelos usuários ou avaliadores.

5.1 Requisitos Mínimos para Executar o Programa em Linux

1. Sistema Operacional derivado do Kernel Linux (Ubuntu 20.04+, Debian, Fedora, Arch, etc.).

2. Interpretador **Python versão 3.8 ou superior** nativamente instalado.
3. Idealmente, firewalls e proteções semelhantes **desligados** para evitar problemas na conexão com o servidor Rendezvous e outros peers.
4. Privilégios regulares de usuário para realizar a abertura (*binding*) de *sockets* nas portas do intervalo dinâmico não privilegiado (portas de 1024 a 65535, configuradas no projeto).

5.2 Compilação do Código

Como o ecossistema é alicerçado sob a linguagem **Python**, não é feito uso de nenhuma linguagem compilada, como C ou Java. Assim, **não deve ser feita nenhuma compilação prévia do código**. O código é interpretado dinamicamente durante o tempo de execução através do interpretador `python3`.

5.3 Bibliotecas Necessárias e Como Instaladas

Apenas a Biblioteca Padrão do Python (Standard Library) foi utilizada. Bibliotecas como `asyncio`, `socket`, `json`, `logging` e `datetime` estão nativamente embarcadas no Python. **Não existem bibliotecas extras exigidas para o uso do programa.**

5.4 Passos Necessários para Executar o Programa

A cadeia de passos sequenciais que guiam o sistema ao pleno funcionamento no terminal Linux é:

1. Faça a abertura de um terminal e navegue via `cd` até o diretório da raiz do código fonte que contém os arquivos Python:

```
$ cd /caminho/ate/Redes_Computadores/
```

2. Opcionalmente, pode-se editar o arquivo de metadados padrão `config.json` para adequar o `namespace`, porta *inbound* e identidade do *peer*.
3. Inicialize a aplicação chamando diretamente o script de partida:

```
$ python3 main.py
```

4. O *CLI* interativo abrirá o canal após notificar seu registro com sucesso. Você pode, agora, explorar os comandos detalhados na Seção 4, e mostrados com o comando `/help`.
5. Digite `/quit` quando terminar de usar, para assim desregistrar-se no servidor e finalizar todas as sessões.

6 Apêndice: Códigos-Fonte

Esta seção contém o código-fonte integral dos módulos que compõem o sistema de Chat P2P do Grupo 2.

6.1 main.py

```
# =====
# PROJETO: Chat P2P (Versão Revisada)
# DISCIPLINA: Redes de Computadores
# INSTITUIÇÃO: Universidade de Brasília (UnB) - CIC
# GRUPO: 2
#
# MEMBROS DA EQUIPE:
# - Gabriel Gonçalves Caldo (Matrícula: 231034627)
# - Daniel Rodrigues de Abreu (Matrícula: 241038540)
# - Mario Augusto Vieira dos Santos (Matrícula: 231035778)
#
# ARQUIVO: main.py
# =====

import asyncio
import logging
from peer_server import PeerServer
from peer_table import PeerTable
from rendezvous_connection import RendezvousConnection
from config import Config
from cli import cli_loop
from logging_config import setup_logger
from reconnection_manager import reconnection_loop

async def main():
    """
    Função principal que configura e inicia a execução da aplicação P2P,
    gerenciando os loops de background para registro e descoberta,
    e garantindo um encerramento limpo (UNREGISTER).
    """
    # Instancia o objeto de configuração
    cfg = Config()
```

```

# Configura o logger
logger = setup_logger(cfg.log_level)
logger.info("Iniciando aplicação Chat P2P...")

# 1. Instanciamento da tabela de peers e state da aplicação
peer_table = PeerTable(max_reconnect_attempts=cfg.
max_reconnect_attempts)
outbound_connections = {}
inbound_connections = {}
shutdown_event = asyncio.Event()

# 2. Criação do servidor do Peer local:
peer_id = f"{cfg.name}@{cfg.namespace}"
port = cfg.listen_port
server = PeerServer(peer_id, port, peer_table, shared_connections=
inbound_connections)

# 3. Inicialização da conexão com o Rendezvous:
rdv_conn = RendezvousConnection(
    cfg.rdv_host,
    cfg.rdv_port
)

# 4. Registro inicial no Rendezvous (síncrono/preparatório)
register_msg = {
    "type": "REGISTER",
    "namespace": cfg.namespace,
    "name": cfg.name,
    "port": port,
    "ttl": cfg.rdv_ttl
}

logger.info("Registrando peer no servidor Rendezvous...")
try:
    response = await rdv_conn.request(register_msg)
    if response.get("status") == "OK":
        granted_ttl = response.get("ttl", cfg.rdv_ttl)

```

```

        logger.info(f"Registro inicial realizado com sucesso! TTL
concedido: {granted_ttl}s")
    else:
        logger.warning(f"Aviso: Servidor Rendezvous retornou erro no
registro: {response.get('message')}")
except Exception as e:
    logger.warning(f"Aviso: Não foi possível conectar ao Rendezvous
para registro inicial: {e}")

# 5. Agendamento dos loops de background (descoberta, re-registro,
reconexão e CLI)

reg_task = asyncio.create_task(
    rdv_conn.registration_loop(
        namespace=cfg.namespace,
        name=cfg.name,
        port=port,
        initial_ttl=cfg.rdv_ttl
    )
)

disc_task = asyncio.create_task(
    rdv_conn.discovery_loop(
        namespace=cfg.namespace,
        peer_table=peer_table,
        interval=cfg.discover_interval,
        exclude_peer_id=peer_id,
        active_connections=outbound_connections,
        local_peer_id=peer_id
    )
)

reconn_task = asyncio.create_task(
    reconnection_loop(
        peer_id=peer_id,
        peer_table=peer_table,
        outbound_connections=outbound_connections
    )
)

```

```

cli_task = asyncio.create_task(
    cli_loop(
        peer_id=peer_id,
        peer_table=peer_table,
        outbound_connections=outbound_connections,
        inbound_connections=inbound_connections,
        shutdown_event=shutdown_event
    )
)

# 6. Execução do servidor local TCP com tratamento de saída limpa
# Inicia o servidor local explicitamente sem bloquear a thread
principal

server_runner = await asyncio.start_server(
    server.handle_client,
    "0.0.0.0",
    server.port,
    limit=32768
)

logger.info(f"Ouvindo conexões (inbound) na porta {server.port}")

try:
    # Aguarda até que o evento de desligamento seja acionado (ex:
    # pelo comando /quit na CLI)
    await shutdown_event.wait()
except asyncio.CancelledError:
    logger.info("Sinal de encerramento recebido (CancelledError).")
finally:
    logger.info("Iniciando encerramento limpo...")

    # Envia BYE para todas conexões de entrada ativas
    for pid, conn in list(inbound_connections.items()):
        if isinstance(conn, str):
            continue

        logger.info(f"Fechando conexão de entrada (inbound) com {pid}
        }...")

        await conn.disconnect(reason="Cliente está sendo desligado (
        shutdown)")

        await conn.close()

```

```

# Envia BYE para todas conexões de saída ativas
for pid, conn in list(outbound_connections.items()):
    if isinstance(conn, str):
        continue
    logger.info(f"Fechando conexão de saída (outbound) com {pid
}...")
    await conn.disconnect(reason="Cliente está sendo desligado (
shutdown)")
    await conn.close()

# Cancela loops assíncronos que estão rodando em background
reg_task.cancel()
disc_task.cancel()
reconn_task.cancel()
cli_task.cancel()

await asyncio.gather(reg_task, disc_task, reconn_task, cli_task,
return_exceptions=True)

# Para o servidor TCP
server_runner.close()
for task in list(server.active_tasks):
    task.cancel()
for w in list(server.active_connections):
    try:
        w.close()
    except Exception:
        pass
await server_runner.wait_closed()

# Envio de UNREGISTER
try:
    await rdv_conn.unregister(
        namespace=cfg.namespace,
        name=cfg.name,
        port=port
    )

```

```
        logger.info("Remoção de registro (UNREGISTER) concluída com  
sucesso.")  
    except Exception as e:  
        logger.error(f"Erro ao tentar remover registro (UNREGISTER):  
{e}")  
  
if __name__ == "__main__":  
    try:  
        asyncio.run(main())  
    except KeyboardInterrupt:  
        pass  
    print("\n[Main] Aplicação encerrada.")
```


6.2 config.py

```
# =====
# PROJETO: Chat P2P (Versão Revisada)
# DISCIPLINA: Redes de Computadores
# INSTITUIÇÃO: Universidade de Brasília (UnB) - CIC
# GRUPO: 2
#
# MEMBROS DA EQUIPE:
# - Gabriel Gonçalves Caldo (Matrícula: 231034627)
# - Daniel Rodrigues de Abreu (Matrícula: 241038540)
# - Mario Augusto Vieira dos Santos (Matrícula: 231035778)
#
# ARQUIVO: config.py
# =====

import json
import sys
import uuid
from pathlib import Path

class Config:
    """
    Classe Singleton para carregar e fornecer configurações do arquivo
    config.json.

    Garante que os arquivos de metadados do projeto sejam lidos uma ú
    nica vez

    na inicialização e compartilhados entre todos os módulos.
    """
    _instance = None

    def __new__(cls):
        # Implementação clássica de um Singleton: se a instância não
        existir, cria-a
        if cls._instance is None:
            cls._instance = super(Config, cls).__new__(cls)
            cls._instance._load_config()
        return cls._instance
```

```

def _load_config(self):
    """Lê o arquivo config.json e extrai as variáveis de ambiente.
    """
    config_path = Path(__file__).parent / 'config.json'

    try:
        with open(config_path, 'r', encoding='utf-8') as f:
            data = json.load(f)

        # Identificação do nó
        self.namespace = data.get("namespace", "Default")
        if not isinstance(self.namespace, str) or len(self.namespace) > 64:
            raise ValueError("Configuração inválida: 'namespace' deve ser string de até 64 caracteres.")

        self.name = data.get("name", f"peer_{uuid.uuid4().hex[:4]}")
        if not isinstance(self.name, str) or len(self.name) > 64:
            raise ValueError("Configuração inválida: 'name' deve ser string de até 64 caracteres.")

        self.peer_id = f"{self.name}@{self.namespace}"

        # Endereço e Porta do Rendezvous (Servidor Central de Descoberta)
        self.rdv_host = data.get("rdv_host", "45.171.101.167")
        self.rdv_port = int(data.get("rdv_port", 8080))
        if not (1 <= self.rdv_port <= 65535):
            raise ValueError("Configuração inválida: 'rdv_port' deve estar na faixa de 1 a 65535.")

        # Configurações do servidor local (porta em que este nó vai escutar conexões)
        self.tcp_port = int(data.get("tcp_port", 0))
        self.listen_port = self.tcp_port # Usando tcp_port como porta de escuta principal
        if not (0 <= self.listen_port <= 65535): # 0 é aceito para alocação dinâmica do sistema operacional nos testes
            raise ValueError("Configuração inválida: 'tcp_port' deve

```

```

    estar na faixa de 0 a 65535.")

    # Protocolo (Metadados do Handshake)
    self.version = data.get("version", "1.0")
    self.features = data.get("features", ["ack"])
    self.fixed_msg_ttl = data.get("fixed_msg_ttl", 1)
    self.type = data.get("type", ["REGISTER", "DISCOVER"])

    # Tempos de reconexão e TTL do Servidor
    self.max_reconnect_attempts = int(data.get("
max_reconnect_attempts", 5))
    self.rdv_ttl = int(data.get("rdv_ttl", 3600))
    if not (1 <= self.rdv_ttl <= 86400):
        raise ValueError("Configuração inválida: 'rdv_ttl' deve
estar na faixa de 1 a 86400 segundos.")

    # Parâmetros adicionais
    self.log_level = data.get("log_level", "INFO")
    self.discover_interval = data.get("discover_interval", 20)
    self.keepalive_interval = data.get("keepalive_interval", 30)

except Exception as e:
    # Em caso de falha severa na leitura do JSON, avisa e
encerra o sistema
    print(f"Erro fatal ao carregar config.json: {e}")
    sys.exit(1)

```

6.3 config.json

```
{
  "app_name": "pyp2p-chat",
  "rdv_host": "rdv.mfcaetano.cc",
  "rdv_port": 8080,
  "listen_host": "0.0.0.0",
  "tcp_port": 6821,
  "discover_interval": 20,
  "keepalive_interval": 30,
  "rdv_ttl": 3600,
  "fixed_msg_ttl": 1,
  "namespace": "CIC",
  "name": "Grupo2",
  "type": [
    "REGISTER",
    "DISCOVER"
  ],
  "log_level": "INFO",
  "features": [
    "ack",
    "metrics"
  ],
  "max_reconnect_attempts": 5
}
```

6.4 logging_config.py

```
# =====
# PROJETO: Chat P2P (Versão Revisada)
# DISCIPLINA: Redes de Computadores
# INSTITUIÇÃO: Universidade de Brasília (UnB) - CIC
# GRUPO: 2
#
# MEMBROS DA EQUIPE:
# - Gabriel Gonçalves Caldo (Matrícula: 231034627)
# - Daniel Rodrigues de Abreu (Matrícula: 241038540)
# - Mario Augusto Vieira dos Santos (Matrícula: 231035778)
#
# ARQUIVO: logging_config.py
# =====

import logging
import sys
import readline

class ReadlineConsoleHandler(logging.Handler):
    """
    Handler customizado de Logging que gerencia a impressão de logs em
    segundo plano
    sem quebrar o prompt visual do usuário gerado pelo 'readline' na CLI
    .

    Quando a CLI está ativa aguardando input (cli_active = True), este
    handler apaga
    a linha atual, imprime o log de rede e então recria o prompt 'p2p> '
    e o que
    o usuário já havia digitado, criando a ilusão de um chat sem
    interrupções.
    """
    cli_active = False # Flag controlada pelo cli.py para saber se
    estamos no prompt
    cli_prompt = "p2p> " # Prompt padrão da CLI

    def emit(self, record):
```

```

        try:
            msg = self.format(record)
            if ReadlineConsoleHandler.cli_active:
                sys.stdout.write('\r\x1b[K')
                sys.stdout.write(msg + '\n')
                sys.stdout.write(ReadlineConsoleHandler.cli_prompt +
readline.get_line_buffer())
                sys.stdout.flush()
                readline.redisplay()
            else:
                sys.stdout.write(msg + '\n')
                sys.stdout.flush()
        except Exception:
            self.handleError(record)

def setup_logger(level_name: str = "INFO") -> logging.Logger:
    """
    Configura o sistema global de logs da aplicação.
    Usa um formato padronizado com timestamp local, nível do log e
mensagem.
    """
    numeric_level = getattr(logging, level_name.upper(), logging.INFO)

    logger = logging.getLogger()
    logger.setLevel(numeric_level)

    # Remove qualquer handler padrão que o Python tenha colocado
    if logger.handlers:
        logger.handlers.clear()

    # Formatação das mensagens: Data Hora | NÍVEL | Mensagem
    formatter = logging.Formatter(
        fmt='%(asctime)s | %(levelname)-8s | %(message)s',
        datefmt='%Y-%m-%d %H:%M:%S'
    )

    # Acopla nosso Handler customizado para lidar com a CLI
    console_handler = ReadlineConsoleHandler()
    console_handler.setLevel(numeric_level)

```

```
console_handler.setFormatter(formatter)

# Tratador de Arquivo (p2p.log)
file_handler = logging.FileHandler("p2p.log", mode='a')
file_handler.setLevel(numeric_level)
file_handler.setFormatter(formatter)

logger.addHandler(console_handler)
logger.addHandler(file_handler)

return logger
```

6.5 cli.py

```
#
=====

# PROJETO: Chat P2P (Versão Revisada)
# DISCIPLINA: Redes de Computadores
# INSTITUIÇÃO: Universidade de Brasília (UnB) - CIC
# GRUPO: 2
#
# MEMBROS DA EQUIPE:
# - Gabriel Gonçalves Caldo (Matrícula: 231034627)
# - Daniel Rodrigues de Abreu (Matrícula: 241038540)
# - Mario Augusto Vieira dos Santos (Matrícula: 231035778)
#
# ARQUIVO: cli.py
#
=====

import asyncio
import sys
import logging
import uuid
import readline # Habilita teclas de seta e histórico para input()
from typing import Dict
from peer_connection import PeerConnection
from peer_table import PeerTable

async def async_input(prompt: str) -> str:
    """
    Lê a entrada do usuário de forma assíncrona usando run_in_executor e
    a função input().
    Como o módulo readline já foi importado, ele automaticamente
    gerencia o histórico de
    comandos e navegação por setas.
    """
    loop = asyncio.get_event_loop()
    # Executa a função síncrona input() em uma thread separada para não
```



```

    bloquear o event loop
    return await loop.run_in_executor(None, input, prompt)

from logging_config import ReadlineConsoleHandler

async def cli_loop(
    peer_id: str,
    peer_table: PeerTable,
    outbound_connections: Dict[str, PeerConnection],
    inbound_connections: Dict[str, PeerConnection],
    shutdown_event: asyncio.Event
):
    """
    Loop interativo da Interface de Linha de Comando (CLI).
    Permite que o usuário insira comandos em tempo real sem interromper
    as rotinas
    de rede que executam em plano de fundo.
    """
    logger = logging.getLogger(__name__)
    await asyncio.sleep(1.0) # Aguarda 1 segundo para garantir que os
    logs iniciais (ex: registro) passem

    ReadlineConsoleHandler.cli_active = False

    while not shutdown_event.is_set():
        try:
            ReadlineConsoleHandler.cli_prompt = "p2p> "
            ReadlineConsoleHandler.cli_active = True
            line = await async_input("p2p> ")
            ReadlineConsoleHandler.cli_active = False
            if shutdown_event.is_set():
                break

            line = line.strip()
            if not line:
                continue

            parts = line.split(" ", 2)
            cmd = parts[0].lower()

```

```

if cmd == "/peers":
    filter_arg = parts[1] if len(parts) > 1 else None
    peers = peer_table.get_all_peers()
    print("--- Peers Conhecidos ---")
    for p in peers:
        if filter_arg == "*" or filter_arg is None:
            pass
        elif filter_arg.startswith("#"):
            if p.namespace != filter_arg[1:]:
                continue
        print(f"[{p.status}] {p.peer_id} em {p.ip}:{p.port} (TTL: {p.ttl})")
    print("-----")

elif cmd == "/msg":
    if len(parts) < 3:
        print("Uso: /msg <peer_id> <mensagem>")
        continue
    target_id = parts[1]
    msg_content = parts[2]

    # Verifica se já possuímos uma conexão TCP ativa com
    este peer

    # Verifica se já possuímos uma conexão TCP ativa com
    este peer (de saída ou de entrada)
    conn = outbound_connections.get(target_id) or
    inbound_connections.get(target_id)

    # Se não possuir, tenta descobrir o peer e estabelecer a
    conexão

    if conn is None:
        peer_entry = peer_table.get_peer(target_id)
        if not peer_entry:
            print(f"Peer desconhecido: {target_id}. Tente
            usar /peers para descobrir novos peers.")
            continue

        if peer_entry.status == "CONNECTED" or target_id in

```

```

outbound_connections or target_id in inbound_connections:
    print(f"Você já possui uma conexão ativa ou em
andamento com o peer: {target_id}")
    continue

    print(f"Conectando a {target_id} em {peer_entry.ip
}:{peer_entry.port}...")
    conn = PeerConnection(
        peer_id,
        peer_table,
        on_close=lambda c: outbound_connections.pop(c.
remote_peer_id, None) if c.remote_peer_id else None
    )
    try:
        await conn.connect(peer_entry.ip, peer_entry.
port)

        # Inicia a tarefa de escuta contínua para essa
nova conexão

        asyncio.create_task(conn.listen())
        outbound_connections[target_id] = conn
    except Exception as e:
        print(f"Falha ao conectar: {e}")
        continue

    # Envia a mensagem com um UUID único
    msg_id = str(uuid.uuid4())
    try:
        await conn.send_message(msg_content, msg_id,
require_ack=True)
    except Exception as e:
        print(f"Falha ao enviar mensagem: {e}")
        if target_id in outbound_connections:
            del outbound_connections[target_id]
        if target_id in inbound_connections:
            del inbound_connections[target_id]

elif cmd == "/pub":
    if len(parts) < 3:
        print("Uso: /pub <* | #namespace> <mensagem>")

```

```

        continue

    scope = parts[1]
    msg_content = parts[2]
    msg_id = str(uuid.uuid4())

    # Identifica todos os peers que correspondem ao escopo
    selecionado (global ou namespace)
    target_peers = []
    for p in peer_table.get_all_peers():
        if scope == "*":
            target_peers.append(p)
        elif scope.startswith("#") and p.namespace == scope
[1:]:
            target_peers.append(p)

    count = 0
    for p in target_peers:
        # Reaproveita a conexão ativa, se já existir (de saída ou de entrada)
        conn = outbound_connections.get(p.peer_id) or
inbound_connections.get(p.peer_id)
        if conn is None:
            if p.status == "CONNECTED" or p.peer_id in
outbound_connections or p.peer_id in inbound_connections:
                continue
            # Se não existir, tenta abrir a conexão em
background para o envio
            conn = PeerConnection(
                peer_id,
                peer_table,
                on_close=lambda c: outbound_connections.pop(
c.remote_peer_id, None) if c.remote_peer_id else None
            )
            try:
                await conn.connect(p.ip, p.port)
                asyncio.create_task(conn.listen())
                outbound_connections[p.peer_id] = conn
            except Exception:
                continue

```

```

        # Dispara a mensagem PUB sobre a conexão, propagando
-a no escopo definido
        try:
            await conn.publish(msg_content, msg_id, scope)
            count += 1
        except Exception:
            pass
    print(f"Broadcast enviado para {count} peer(s).")

elif cmd == "/conn":
    print("--- Conexões Ativas de Entrada (Inbound) ---")
    for pid, conn in inbound_connections.items():
        print(f"{pid} -> Connected")
    print("--- Conexões Ativas de Saída (Outbound) ---")
    # Exibe uma lista das conexões TCP que foram iniciadas
por este nó
    for pid, conn in outbound_connections.items():
        print(f"{pid} -> Connected")
    print("-----")

elif cmd == "/rtt":
    print("--- RTT Médio por Peer ---")
    for p in peer_table.get_all_peers():
        if p.average_rtt is not None:
            print(f"{p.peer_id}: {p.average_rtt:.2f} ms")
        else:
            print(f"{p.peer_id}: N/D")
    print("-----")

elif cmd == "/reconnect":
    print("Iniciando reconexão manual para todos os peers
desconectados/obsoletos...")
    for p in peer_table.get_all_peers():
        if p.status in ["DISCONNECTED", "STALE", "
RECONNECTING"]:
            p.status = "RECONNECTING"
            p.next_attempt_allowed_at = 0.0 # Força
tentativa imediata

```

```

        p.reconnect_attempts = 0

    elif cmd == "/log":
        if len(parts) < 2:
            print("Uso: /log <DEBUG|INFO|WARNING|ERROR>")
            continue

        level_str = parts[1].upper()
        num_level = getattr(logging, level_str, None)
        if num_level is not None:
            logging.getLogger().setLevel(num_level)
            for handler in logging.getLogger().handlers:
                handler.setLevel(num_level)
            print(f"Nível de log alterado para {level_str}")
        else:
            print(f"Nível de log inválido: {level_str}")

    elif cmd == "/help":
        print("--- Comandos Disponíveis ---")
        print("  /help                                :  
Exibe esta mensagem de ajuda")
        print("  /peers [* | #namespace]              :  
Lista peers conhecidos")
        print("  /msg <peer_id> <mensagem>            :  
Envia mensagem direta (unicast)")
        print("  /pub <*> | #namespace> <mensagem>    :  
Envia mensagem global (broadcast)")
        print("  /conn                                  :  
Exibe conexões ativas (saída)")
        print("  /rtt                                   :  
Exibe o RTT médio por peer")
        print("  /reconnect                             : Força  
a tentativa imediata de reconexão")
        print("  /log <DEBUG|INFO|WARNING|ERROR>      :  
Altera nível do log do sistema")
        print("  /quit                                  : Sai  
e finaliza a aplicação")
        print("-----")

    elif cmd == "/quit":

```

```
        print("Iniciando encerramento limpo...")
        shutdown_event.set()
        break

    else:
        print(f"Comando desconhecido: {cmd}. Digite /help para
listar comandos.")

except Exception as e:
    logger.error(f"Erro no CLI: {e}")
    break

# Sinaliza que o CLI foi desativado para restaurar o comportamento
normal de logs
ReadlineConsoleHandler.cli_active = False
```

6.6 message_router.py

```
# =====
# PROJETO: Chat P2P (Versão Revisada)
# DISCIPLINA: Redes de Computadores
# INSTITUIÇÃO: Universidade de Brasília (UnB) - CIC
# GRUPO: 2
#
# MEMBROS DA EQUIPE:
# - Gabriel Gonçalves Caldo (Matrícula: 231034627)
# - Daniel Rodrigues de Abreu (Matrícula: 241038540)
# - Mario Augusto Vieira dos Santos (Matrícula: 231035778)
#
# ARQUIVO: message_router.py
# =====

import json
import uuid
import datetime
import logging
from config import Config

def validate_message_fields(msg: dict):
    """
    Valida os campos de uma mensagem de acordo com as especificações do
    protocolo:

    - namespace: string (até 64 caracteres)
    - name: string (até 64 caracteres)
    - port: inteiro (1-65535)
    - ttl: inteiro em segundos (1-86400)

    Lança ValueError caso alguma validação falhe.
    """
    # Validação de namespace
    if "namespace" in msg:
        ns = msg["namespace"]
        if not isinstance(ns, str):
            raise ValueError("O campo 'namespace' deve ser uma string.")
        if len(ns) > 64:
```



```
        raise ValueError("O campo 'namespace' não deve exceder 64
caracteres.")

# Validação de name
if "name" in msg:
    name = msg["name"]
    if not isinstance(name, str):
        raise ValueError("O campo 'name' deve ser uma string.")
    if len(name) > 64:
        raise ValueError("O campo 'name' não deve exceder 64
caracteres.")

# Validação de peer_id (se contiver name@namespace)
if "peer_id" in msg:
    pid = msg["peer_id"]
    if not isinstance(pid, str):
        raise ValueError("O campo 'peer_id' deve ser uma string.")
    if "@" in pid:
        parts = pid.split("@", 1)
        name_part = parts[0]
        ns_part = parts[1]
        if len(name_part) > 64:
            raise ValueError("O nome extraído de 'peer_id' não deve
exceder 64 caracteres.")
        if len(ns_part) > 64:
            raise ValueError("O namespace extraído de 'peer_id' não
deve exceder 64 caracteres.")
        else:
            if len(pid) > 64:
                raise ValueError("O campo 'peer_id' não deve exceder 64
caracteres.")

# Validação de port
if "port" in msg:
    port = msg["port"]
    try:
        port_val = int(port)
    except (ValueError, TypeError):
        raise ValueError("O campo 'port' deve ser um inteiro válido.
```

```

    ")

    if not (1 <= port_val <= 65535):
        raise ValueError(f"O campo 'port' deve estar entre 1 e 65535. Valor recebido: {port_val}")

    # Validação de ttl
    if "ttl" in msg:
        ttl = msg["ttl"]
        try:
            ttl_val = int(ttl)
        except (ValueError, TypeError):
            raise ValueError("O campo 'ttl' deve ser um inteiro válido.")
    )

    if not (1 <= ttl_val <= 86400):
        raise ValueError(f"O campo 'ttl' deve estar entre 1 e 86400. Valor recebido: {ttl_val}")

def encode_message(data : dict) -> bytes:
    """
    Transforma um dicionário Python em uma string JSON codificada em bytes,
    adicionando uma quebra de linha ('\n') no final para servir de delimitador.
    Também valida os formatos dos campos da mensagem antes de enviar.
    """
    validate_message_fields(data)
    return (json.dumps(data) + "\n").encode('utf-8')

def decode_message(raw : bytes) -> dict:
    """
    Decodifica uma sequência de bytes recebida da rede (que contém uma string JSON)
    e reconstrói o dicionário Python original, validando seus campos.
    """
    msg = json.loads(raw.decode('utf-8'))
    validate_message_fields(msg)
    return msg

async def process_common_messages(msg: dict, local_peer_id: str,

```

```

remote_peer_id: str, send_func) -> str:
    """
    Processa mensagens comuns do protocolo (PING, SEND, PUB, BYE) que
    podem chegar
    tanto pelo canal de servidor (inbound) quanto pelo canal de cliente
    (outbound).

    :param msg: Dicionário decodificado da mensagem.
    :param local_peer_id: Identificador local (ex: alice@CIC).
    :param remote_peer_id: Identificador remoto.
    :param send_func: Função/Corrotina async que recebe um dicionário
    para enviá-lo de volta.
    :return: "BREAK" se a conexão deve ser encerrada, "CONTINUE" se deve
    ser repassada, "HANDLED" se tratada.
    """

    msg_type = msg.get("type")

    if msg_type == "PING":
        logging.getLogger(__name__).debug(f"PING RECEBIDO de {
remote_peer_id or 'desconhecido'}")
        pong = {
            "type": "PONG",
            "msg_id": msg.get("msg_id", str(uuid.uuid4())),
            "timestamp": datetime.datetime.utcnow().isoformat() + "Z",
            "ttl": Config().fixed_msg_ttl
        }
        await send_func(pong)
        logging.getLogger(__name__).debug(f"PONG ENVIADO para {
remote_peer_id or 'desconhecido'}")
        return "HANDLED"

    elif msg_type == "SEND":
        sender = msg.get("src", remote_peer_id or "desconhecido")
        content = msg.get("payload", "")
        msg_id = msg.get("msg_id")
        logging.getLogger(__name__).info(f"[SEND] {sender}: {content}")

        if msg.get("require_ack") and msg_id:
            ack = {

```

```

        "type": "ACK",
        "msg_id": msg_id,
        "timestamp": datetime.datetime.utcnow().isoformat() + "Z
",
        "ttl": Config().fixed_msg_ttl
    }
    await send_func(ack)
    logging.getLogger(__name__).debug(f"[ACK] Recibo ACK enviado
para msg_id={msg_id} (destino: {sender})")
    return "HANDLED"

elif msg_type == "PUB":
    sender = msg.get("src", remote_peer_id or "desconhecido")
    content = msg.get("payload", "")
    scope = msg.get("dst", "*")
    logging.getLogger(__name__).info(f"[PUB] [{scope}] {sender}: {
content}")
    return "HANDLED"

elif msg_type == "BYE":
    reason = msg.get("reason", "Sem motivo especificado")
    logging.getLogger(__name__).info(f"Recebido comando BYE de {
remote_peer_id or 'desconhecido'}. Motivo: {reason}")

    bye_ok = {
        "type": "BYE_OK",
        "msg_id": msg.get("msg_id", str(uuid.uuid4())),
        "src": local_peer_id,
        "dst": msg.get("src", remote_peer_id or "desconhecido"),
        "ttl": Config().fixed_msg_ttl
    }
    await send_func(bye_ok)
    return "BREAK"

return "CONTINUE"

```

6.7 peer_server.py

```
# =====
# PROJETO: Chat P2P (Versão Revisada)
# DISCIPLINA: Redes de Computadores
# INSTITUIÇÃO: Universidade de Brasília (UnB) - CIC
# GRUPO: 2
#
# MEMBROS DA EQUIPE:
# - Gabriel Gonçalves Caldo (Matrícula: 231034627)
# - Daniel Rodrigues de Abreu (Matrícula: 241038540)
# - Mario Augusto Vieira dos Santos (Matrícula: 231035778)
#
# ARQUIVO: peer_server.py
# =====

import asyncio
import logging
from typing import Optional
from peer_table import PeerTable
# Nota Importante: O import 'writer' do módulo csv não é utilizado neste
# arquivo. Ele provavelmente
# foi importado por engano no lugar do StreamWriter do asyncio. Mantemos
# aqui por questões de compatibilidade
# com o rascunho inicial do usuário, mas adicionamos esse alerta.
from message_router import decode_message, encode_message,
    process_common_messages
from config import Config
import uuid
import datetime
import sys
import readline

class PeerServer:
    """
    Representa o servidor TCP local deste nó (Peer).

    O que este servidor faz?
    Ele abre uma porta TCP local em seu computador e fica escutando em
```

```

segundo plano. Quando outros peers
na rede (ex: Alice ou Bob) descobrem você através do Rendezvous,
eles usam o 'PeerConnection' deles
para se conectar à porta do seu 'PeerServer'.
Esta classe gerencia a recepção e resposta de handshakes (HELLO) e
mensagens de controle (PING/PONG).
"""

def __init__(self, peer_id, port, peer_table: Optional[PeerTable] =
None, shared_connections: Optional[dict] = None):
    # Identificador do peer local (ex: 'alice@CIC'). Usado para se
    # apresentar a outros peers no handshake.
    self.peer_id = peer_id
    # Porta de rede local onde o servidor irá escutar conexões de
    # entrada (ex: 5000)
    self.port = port
    # Tabela de peers compartilhada para gerenciar os estados das
    # conexões
    self.peer_table = peer_table
    # Dicionário de conexões ativas compartilhado com o cliente/CLI
    self.shared_connections = shared_connections
    # Conexões ativas de clientes (inbound)
    self.active_connections = set()
    # Tarefas ativas de clientes (inbound)
    self.active_tasks = set()
    # Mensagens recebidas (histórico de SEND/PUB) para
    # observabilidade e testes
    self.received_messages = []

    async def handle_client(self, reader: asyncio.StreamReader, writer:
    asyncio.StreamWriter):
        """
        Método de callback assíncrono que o asyncio executa
        automaticamente toda vez que um novo peer se conecta.

        Como funciona a concorrência assíncrona aqui?
        Para cada cliente que se conecta, o asyncio cria uma nova tarefa
        e executa este método em segundo plano.
        Isso permite que múltiplos peers conversem com você

```

simultaneamente sem que um bloqueie o outro.

Parâmetros:

 reader: Fluxo de leitura assíncrono (StreamReader) para receber dados enviados por este peer.

 writer: Fluxo de escrita assíncrono (StreamWriter) para enviar dados de volta a este peer.

```
"""
remote_peer = None
self.active_connections.add(writer)
current_task = asyncio.current_task()
if current_task:
    self.active_tasks.add(current_task)
try:
    # 1. O primeiro pacote deve ser obrigatoriamente um HELLO de
    handshake
    try:
        data = await asyncio.wait_for(reader.readline(), timeout
=5.0)

    except asyncio.TimeoutError:
        logging.getLogger(__name__).warning("[PeerServer]
Timeout de 5s aguardando HELLO. Fechando conexão.")
        return
    if not data:
        return

    msg = decode_message(data)

    if msg.get("type") != "HELLO":
        logging.getLogger(__name__).warning("[PeerServer] Conexã
o rejeitada: primeira mensagem não é HELLO")
        return

    remote_peer = msg.get("peer_id", "desconhecido")
    remote_addr = writer.get_extra_info('peername')
    logging.getLogger(__name__).info(f"[PeerServer] Conexão de
entrada de {remote_peer} em {remote_addr}")

    # --- EVITAR CONEXÕES DUPLICADAS ---
```

```

        # if self.shared_connections is not None and remote_peer in
self.shared_connections:

        #     existing_conn = self.shared_connections[remote_peer]
        #     if existing_conn != "connecting":
        #         logging.getLogger(__name__).warning(f"[PeerServer]
Conexão duplicada detectada para {remote_peer}. Rejeitando nova
conexão.")
        #         return

        # Registra o peer na tabela
        if self.peer_table:
            self.peer_table.update_peer(remote_peer, remote_addr[0],
remote_addr[1], msg.get("ttl", 3600))

        # Envia HELLO_OK
        hello_ok = {
            "type": "HELLO_OK",
            "peer_id": self.peer_id,
            "status": "OK",
            "features": Config().features,
            "ttl": Config().fixed_msg_ttl
        }
        writer.write(encode_message(hello_ok))
        await writer.drain()

        # Delega o gerenciamento e a escuta da conexão para a classe
PeerConnection
        if self.shared_connections is not None:
            from peer_connection import PeerConnection
            inbound_conn = PeerConnection(self.peer_id, self.
peer_table)

            inbound_conn.reader = reader
            inbound_conn.writer = writer
            inbound_conn.remote_peer_id = remote_peer

            # Agenda envio periódico de keep-alive (PING) para o
peer

            inbound_conn.keepalive_task = asyncio.create_task(
inbound_conn.keepalive_loop())

```



```

        self.shared_connections[remote_peer] = inbound_conn

    try:
        await inbound_conn.listen()
    finally:
        self.shared_connections.pop(remote_peer, None)

except Exception as e:
    logging.getLogger(__name__).error(f"Erro no servidor ao lidar com cliente {remote_peer or 'desconhecido'}: {e}")
finally:
    self.active_connections.discard(writer)
    current_task = asyncio.current_task()
    if current_task:
        self.active_tasks.discard(current_task)
    # Garante que o socket seja fechado de forma limpa
    try:
        writer.close()
        await writer.wait_closed()
    except Exception:
        pass

    logging.getLogger(__name__).info(f"Conexão com cliente {remote_peer or 'desconhecido'} fechada")

    # Atualiza o status do peer na tabela se ele estava conectado
    if self.peer_table and remote_peer:
        peer_entry = self.peer_table.get_peer(remote_peer)
        if peer_entry and peer_entry.status == "CONNECTED":
            self.peer_table.mark_disconnected(remote_peer)

async def start(self):
    """
    Inicializa e inicia o servidor TCP local, colocando-o em estado de escuta (listening).

    Como funciona:
    1. 'asyncio.start_server' cria o socket TCP do sistema operacional e o vincula (bind) à porta especificada.

```

2. O endereço '0.0.0.0' significa que o servidor aceitará conexões vindas de qualquer placa de rede

disponível no seu computador (tanto localhost/127.0.0.1 quanto conexões de rede externa).

3. Registramos o 'self.handle_client' como a corrotina que cuidará de cada conexão.

```
"""
server = await asyncio.start_server(
    self.handle_client,
    "0.0.0.0",
    self.port
)

logging.getLogger(__name__).info(f"Escutando na porta {self.port}")

try:
    # Mantém o loop de eventos focado em servir o socket deste
    servidor para sempre,
    # impedindo que o programa termine e pare de aceitar conexões.

    await asyncio.Event().wait()
finally:
    server.close()
    logging.getLogger(__name__).info(f"[PeerServer] Parando
    servidor na porta {self.port}, cancelando {len(self.active_tasks)}
    tarefas de clientes ativos...")
    for task in list(self.active_tasks):
        task.cancel()
    for w in list(self.active_connections):
        try:
            w.close()
        except Exception:
            pass
    await server.wait_closed()
```

6.8 peer_connection.py

```
# =====
# PROJETO: Chat P2P (Versão Revisada)
# DISCIPLINA: Redes de Computadores
# INSTITUIÇÃO: Universidade de Brasília (UnB) - CIC
# GRUPO: 2
#
# MEMBROS DA EQUIPE:
# - Gabriel Gonçalves Caldo (Matrícula: 231034627)
# - Daniel Rodrigues de Abreu (Matrícula: 241038540)
# - Mario Augusto Vieira dos Santos (Matrícula: 231035778)
#
# ARQUIVO: peer_connection.py
# =====

import asyncio
import time
import logging
from typing import Optional, Dict
from message_router import decode_message, encode_message,
    process_common_messages
from config import Config
import uuid
import datetime
import sys
import readline

class PeerConnection:
    """
    Gerencia uma conexão de rede TCP ativa de saída do seu cliente para
    um peer remoto.

    O que esta classe faz?
    Quando o seu programa descobre outro peer ativo (ex: Bob) e você
    deseja falar com ele, você instancia
    um 'PeerConnection'. Ele é responsável por:
    1. Abrir a conexão TCP física ('asyncio.open_connection').
    2. Realizar o handshake do protocolo enviando a mensagem 'HELLO' e
```

esperando 'HELLO_OK'.

3. Manter a escuta em segundo plano para capturar as mensagens de chat que aquele peer te enviar.

"""

```
def __init__(self, peer_id, peer_table=None, on_close=None):
    # 0 identificador único do peer local (ex: 'Grupo2@CIC')
    self.peer_id = peer_id

    # Tabela de peers compartilhada
    self.peer_table = peer_table

    # Callback executado ao fechar a conexão
    self.on_close = on_close

    # Fluxo de entrada assíncrono para ler dados do socket do peer
    remoto
    self.reader: Optional[asyncio.StreamReader] = None

    # Fluxo de saída assíncrono para escrever dados no socket do
    peer remoto
    self.writer: Optional[asyncio.StreamWriter] = None

    # 0 ID do peer remoto ao qual nos conectamos (ex: 'bob@CIC')
    # É preenchido de forma dinâmica durante o handshake após
    receber o HELLO_OK
    self.remote_peer_id = None

    # Controle de Keep-Alive e RTT
    self.keepalive_task: Optional[asyncio.Task] = None
    self.last_ping_time: Optional[float] = None
    self.keepalive_interval = 30

    # Mapa de ACKs pendentes: msg_id -> asyncio.Event
    # Quando enviamos uma mensagem com require_ack=True, registramos
    um Event aqui.
    # 0 loop de escuta sinaliza o Event quando o ACK correspondente
    chegar.
    self._pending_acks: Dict[str, asyncio.Event] = {}
```

```

        # Timeout (em segundos) para aguardar um ACK antes de emitir
aviso nos logs
        self.ack_timeout = 5.0

async def send(self, msg: dict):
    """
        Codifica e envia uma mensagem JSON estruturada pelo canal de saí
da do socket.
    """
    # Escreve os bytes serializados da mensagem (terminada em \n) no
buffer interno
    self.writer.write(
        encode_message(msg)
    )
    # O drain() é uma corrotina assíncrona que esvazia o buffer de
rede.
    # Ele suspende esta função temporariamente até que todos os
bytes tenham sido gravados
    # fisicamente na placa de rede, garantindo que a mensagem foi de
fato transmitida.
    await self.writer.drain()

async def listen(self):
    """
        Loop contínuo de escuta executado em background para receber
dados enviados por este peer.

        Como funciona:
        Uma vez estabelecida a conexão, esta função roda infinitamente
em uma tarefa em segundo plano.
        Ela lê cada linha terminada em '\n' enviada pelo peer remoto,
decodifica e imprime o conteúdo.
        Se a conexão cair ou houver timeout, o loop é interrompido e a
limpeza de recursos é feita.
    """
    try:
        while True:
            # Lê bytes da rede com timeout

```

```

        try:
            data = await asyncio.wait_for(
                self.reader.readline(),
                timeout=self.keepalive_interval * 2
            )
        except asyncio.TimeoutError:
            logging.getLogger(__name__).warning(f"Tempo limite
de conexão com {self.remote_peer_id}")
            break

        # Se 'data' for vazio, o peer remoto fechou a conexão de
        forma limpa.
        # Encerramos o loop imediatamente.
        if not data:
            logging.getLogger(__name__).info(f"Conexão fechada
pelo peer remoto: {self.remote_peer_id}")
            break

        # Desserializa os bytes binários para um dicionário JSON
        legível do Python
        msg = decode_message(data)

        action = await process_common_messages(msg, self.peer_id
, self.remote_peer_id, self.send)
        if action == "BREAK":
            break
        elif action == "HANDLED":
            continue

        # Processamento de Keep-Alive e confirmações (Específico
do Cliente)
        if msg.get("type") == "PONG":
            if self.last_ping_time is not None:
                rtt_ms = (time.time() - self.last_ping_time) *
1000

                logging.getLogger(__name__).debug(f"[RTT] RTT
medido para {self.remote_peer_id}: {rtt_ms:.2f} ms")
                if self.peer_table and self.remote_peer_id:
                    peer_entry = self.peer_table.get_peer(self.

```

```

remote_peer_id)

        if peer_entry:
            peer_entry.add_rtt(rtt_ms)

    elif msg.get("type") == "ACK":
        msg_id = msg.get("msg_id")
        if msg_id and msg_id in self._pending_acks:
            self._pending_acks[msg_id].set()
            logging.getLogger(__name__).debug(f"[ACK] ACK
recebido para msg_id={msg_id} de {self.remote_peer_id}")
        else:
            logging.getLogger(__name__).debug(f"[ACK] ACK
inesperado recebido (msg_id={msg_id}) de {self.remote_peer_id}")

    elif msg.get("type") == "BYE_OK":
        logging.getLogger(__name__).info(f"Recebido BYE_OK
de {self.remote_peer_id}")
        break

    else:
        # Exibe no terminal qualquer outra mensagem
        estruturada recebida
        logging.getLogger(__name__).info(f"Mensagem
estruturada desconhecida recebida: {msg}")
    except Exception as e:
        logging.getLogger(__name__).error(f"Erro de conexão com {
self.remote_peer_id}: {e}")
    finally:
        await self.close()
        if self.peer_table and self.remote_peer_id:
            # Atualiza estado para DISCONNECTED
            peer_entry = self.peer_table.get_peer(self.
remote_peer_id)
            if peer_entry and peer_entry.status == "CONNECTED":
                self.peer_table.mark_disconnected(self.
remote_peer_id)

    async def connect(self, host: str, port: int):

```

```

"""
    Conecta-se ativamente ao IP e porta informados, executando o
    handshake de apresentação P2P.
"""

try:
    # Abre a conexão TCP de forma assíncrona. Retorna o leitor e
    o escritor do socket.

    self.reader, self.writer = await asyncio.wait_for(
        asyncio.open_connection(host, port, limit=32768),
        timeout=5.0
    )

    logging.getLogger(__name__).debug("Conexão TCP física
    estabelecida, iniciando handshake...")

    # Prepara a mensagem de identificação 'HELLO' conforme os
    padrões do protocolo
    hello_msg = {
        "type": "HELLO",
        "peer_id": self.peer_id,
        "version": "1.0",
        "features": Config().features,
        "ttl": Config().fixed_msg_ttl
    }

    # Prepara uma mensagem inicial de PING de keep-alive
    ping_msg = {
        "type": "PING",
        "msg_id": str(uuid.uuid4()),
        "timestamp": datetime.datetime.utcnow().isoformat() + "Z
",
        "ttl": Config().fixed_msg_ttl
    }

    # Envia a apresentação (HELLO)
    await self.send(hello_msg)

    # Envia o teste de atividade (PING)
    self.last_ping_time = time.time()

```



```

        await self.send(ping_msg)

        # Aguarda a transmissão física de ambos os pacotes
        await self.writer.drain()

        logging.getLogger(__name__).debug("HELLO enviado")

        # O programa fica pausado de forma assíncrona aguardando a
        resposta do peer remoto
        data = await asyncio.wait_for(self.reader.readline(),
        timeout=5.0)

        # Converte a resposta em dicionário Python
        msg = decode_message(data)

        logging.getLogger(__name__).debug(f"Resposta do Handshake: {
msg}")

        # Validação do Handshake:
        # Se o peer remoto responder com qualquer coisa diferente de
        HELLO_OK,
        # a conexão é considerada inválida pela regra do protocolo e
        lançamos um erro.
        if msg["type"] != "HELLO_OK":
            raise Exception(
                f"Esperado HELLO_OK, recebido {msg['type']}")
        )

        # Salva o identificador (ex: 'bob@CIC') que o peer remoto
        informou no seu HELLO_OK
        self.remote_peer_id = msg["peer_id"]

        logging.getLogger(__name__).info(f"Conectado a {self.
        remote_peer_id}")

        # Se a tabela de peers existe, marca como conectado
        if self.peer_table:
            self.peer_table.mark_connected(self.remote_peer_id)

```

```

        # Agenda a execução recorrente de PINGs em background
        self.keepalive_task = asyncio.create_task(self.
keepalive_loop())
    except Exception:
        await self.close()
        raise

async def send_message(self, content: str, msg_id: str, require_ack:
bool = False):
    """
    Envia uma mensagem unicast do tipo SEND ao peer remoto.

    Parâmetros:
        content      : Texto da mensagem a ser enviada.
        msg_id       : Identificador único da mensagem (ex: UUID).
        require_ack  : Se True, aguarda um ACK do destinatário por at
é 'ack_timeout' segundos.

        Caso o ACK não chegue dentro do prazo, emite um
aviso nos logs.

    Fluxo com require_ack=True:
    1. Um asyncio.Event é registrado em '_pending_acks' antes do
envio.
    2. A mensagem SEND é transmitida com o campo 'require_ack': True
.
    3. Aguardamos o Event ser sinalizado pelo loop de escuta quando
o ACK chegar.
    4. Se expirar o timeout, registramos o aviso e limpamos o Event
pendente.
    """
    msg = {
        "type": "SEND",
        "msg_id": msg_id,
        "src": self.peer_id,
        "dst": self.remote_peer_id,
        "payload": content,
        "require_ack": require_ack,
        "ttl": Config().fixed_msg_ttl
    }

```

```

        if require_ack:
            # Registra o evento pendente ANTES de enviar para evitar
            race condition
            ack_event = asyncio.Event()
            self._pending_acks[msg_id] = ack_event

        await self.send(msg)
        logging.getLogger(__name__).debug(f"[SEND] Mensagem enviada para
        {self.remote_peer_id} (msg_id={msg_id}, require_ack={require_ack})")

        if require_ack:
            try:
                await asyncio.wait_for(ack_event.wait(), timeout=self.
                ack_timeout)
            except asyncio.TimeoutError:
                logging.getLogger(__name__).warning(
                    f"[ACK] AVISO: Nenhum ACK recebido de {self.
                    remote_peer_id} "
                    f"para msg_id={msg_id} dentro de {self.ack_timeout}s
                    "
                )
            finally:
                # Garante limpeza do mapa independente do resultado
                self._pending_acks.pop(msg_id, None)

    async def publish(self, content: str, msg_id: str, scope: str):
        """
        Envia uma mensagem de difusão (broadcast) do tipo PUB a este
        peer remoto.

        O campo 'dst' indica o escopo da difusão:
            - '#namespace' : difusão para todos os peers do namespace
            atual.
            - '*'           : difusão global para todos os peers
            conectados.

        Parâmetros:
            content : Texto da mensagem a ser difundida.

```

`msg_id` : Identificador único da mensagem (ex: UUID).
`scope` : '#namespace' ou '*', conforme especificação do protocolo.

Nota: Este método transmite a mensagem a UM peer por vez. O remetente é

responsável por chamar 'publish()' em cada PeerConnection ativa para realizar

a difusão completa (ver lógica de fan-out no nível da aplicação)

```
"""  
msg = {  
    "type": "PUB",  
    "msg_id": msg_id,  
    "src": self.peer_id,  
    "dst": scope,  
    "payload": content,  
    "require_ack": False,  
    "ttl": Config().fixed_msg_ttl  
}  
await self.send(msg)  
logging.getLogger(__name__).debug(f"[PUB] Broadcast enviado para  
{self.remote_peer_id} (scope={scope}, msg_id={msg_id})")  
  
async def keepalive_loop(self):  
    """  
    Loop periódico em background para enviar PINGs ao peer remoto.  
    """  
    try:  
        while True:  
            await asyncio.sleep(self.keepalive_interval)  
            ping_msg = {  
                "type": "PING",  
                "msg_id": str(uuid.uuid4()),  
                "timestamp": datetime.datetime.utcnow().isoformat()  
+ "Z",  
                "ttl": Config().fixed_msg_ttl  
            }  
            self.last_ping_time = time.time()
```

```

        await self.send(ping_msg)
        logging.getLogger(__name__).debug(f"[KeepAlive] PING
enviado para {self.remote_peer_id}")
    except asyncio.CancelledError:
        pass
    except Exception as e:
        logging.getLogger(__name__).error(f"[KeepAlive] Erro ao
enviar PING para {self.remote_peer_id}: {e}")

async def disconnect(self, reason: str = "Cliente fechando conexão")
:
    """
    Envia uma mensagem de BYE para fechar a conexão de forma limpa.
    """
    bye_msg = {
        "type": "BYE",
        "msg_id": str(uuid.uuid4()),
        "src": self.peer_id,
        "dst": self.remote_peer_id,
        "reason": reason,
        "ttl": Config().fixed_msg_ttl
    }
    try:
        await self.send(bye_msg)
        logging.getLogger(__name__).debug(f"[BYE] Enviado para {self
.remote_peer_id} com motivo: {reason}")
    except Exception as e:
        logging.getLogger(__name__).error(f"[BYE] Falha ao enviar
BYE para {self.remote_peer_id}: {e}")

async def close(self):
    """
    Fecha a conexão TCP e cancela tarefas de background.
    """
    if self.keepalive_task:
        self.keepalive_task.cancel()
        self.keepalive_task = None

    if self.writer:

```

```
        try:
            self.writer.close()
            await self.writer.wait_closed()
        except Exception:
            pass

        self.writer = None
    self.reader = None
    if self.on_close:
        try:
            self.on_close(self)
        except Exception:
            pass
```

6.9 peer_table.py

```
# =====
# PROJETO: Chat P2P (Versão Revisada)
# DISCIPLINA: Redes de Computadores
# INSTITUIÇÃO: Universidade de Brasília (UnB) - CIC
# GRUPO: 2
#
# MEMBROS DA EQUIPE:
# - Gabriel Gonçalves Caldo (Matrícula: 231034627)
# - Daniel Rodrigues de Abreu (Matrícula: 241038540)
# - Mario Augusto Vieira dos Santos (Matrícula: 231035778)
#
# ARQUIVO: peer_table.py
# =====

import time
from typing import Dict, List, Optional
from config import Config

class PeerEntry:
    """
    Representa o estado e todas as informações de rede e conexão de um
    peer remoto específico.

    Cada peer que você descobre ou se conecta possui uma instância única
    desta classe na sua tabela.
    """
    def __init__(self, name: str, namespace: str, ip: str, port: int,
ttl: int):
        # Nome identificador do usuário (ex: 'alice', 'bob')
        self.name = name

        # Compartimento lógico onde o usuário se encontra (ex: 'CIC', '
UnB')
        self.namespace = namespace

        # Identificador composto único para este peer na rede (ex: '
alice@CIC')
        # Este ID é usado como a chave primária para buscar contatos na
```

```

tabela

    self.peer_id = f"{name}@{namespace}"

    # Endereço IP físico do computador remoto (ex: '192.168.1.15' ou
    '45.171.101.167')
    self.ip = ip

    # Porta de rede TCP na qual o servidor deste peer está escutando
    conexões
    self.port = port

    # Time-to-live: Tempo em segundos que este cadastro no servidor
    Rendezvous permanece válido
    self.ttl = ttl

    # Instante de tempo do sistema (timestamp Unix) em que este
    cadastro expira e
    # deve ser considerado inválido ou removido.
    self.expires_at = time.time() + ttl

    # Estado atual da sua conexão TCP com este peer. Pode assumir um
    de quatro valores:
    # - "DISCONNECTED": Estado inicial, você conhece o peer mas não
    há conexão TCP ativa com ele.
    # - "CONNECTED": Existe um socket TCP aberto, handshake (HELLO)
    feito com sucesso e keep-alive ativo.
    # - "RECONNECTING": Uma conexão anterior caiu e o peer está
    aguardando o tempo de backoff para tentar novamente.
    # - "STALE": O peer falhou em responder todas as tentativas de
    conexão e foi marcado como inativo/offline.
    self.status = "DISCONNECTED"

    # Lista contendo os últimos valores medidos de RTT (tempo de ida
    e volta) em milissegundos.
    # Funciona como uma fila circular que retém apenas as medições
    mais recentes.
    self.rtt_history: List[float] = []

    # Média dos RTTs medidos (None se nenhuma medição foi feita

```



```

ainda)

    self.average_rtt: Optional[float] = None

    # Contador de quantas tentativas de reconexão TCP seguidas
    falharam para este peer.
    # É zerado assim que a conexão é estabelecida com sucesso.
    self.reconnect_attempts = 0

    # Timestamp Unix que guarda o momento exato em que a última
    tentativa de conexão falhou.
    self.last_attempt_time = 0.0

    # Timestamp Unix que diz a partir de qual momento o seu programa
    está autorizado
    # a tentar se conectar a este peer novamente (usado para
    respeitar o tempo de backoff).
    self.next_attempt_allowed_at = 0.0

def update_info(self, ip: str, port: int, ttl: int):
    """
    Atualiza as informações de rede do peer quando ele é retornado
    em um novo DISCOVER do Rendezvous.

    Por que é necessário?
    Os peers podem mudar de IP dinamicamente (ex: mudar de rede Wi-
    Fi) ou reiniciar seu contador
    de expiração (TTL). Esse método garante que seu programa use os
    dados atualizados.
    """
    self.ip = ip
    self.port = port
    self.ttl = ttl

    # Recalcula a hora de expiração com base no novo TTL recebido
    self.expires_at = time.time() + ttl

def add_rtt(self, rtt_ms: float):
    """
    Registra uma nova medição de tempo de resposta (RTT) obtida por
    uma troca de PING/PONG.

```

```

        Para evitar consumo excessivo de memória, mantemos apenas as últimas 10 medições na lista.
        """
        self.rtt_history.append(rtt_ms)

        # Se ultrapassar 10 medições, removemos a mais antiga (primeiro elemento da lista)
        if len(self.rtt_history) > 10:
            self.rtt_history.pop(0)

        # Calcula e atualiza a média do RTT diretamente na variável
        self.average_rtt = sum(self.rtt_history) / len(self.rtt_history)

```

```

class PeerTable:

```

```

    """
    Gerencia o banco de dados local na memória contendo a lista de todos os peers conhecidos
    e centraliza a lógica de controle de estados, contagem de reconexões e backoff exponencial.
    """

    def __init__(self, max_reconnect_attempts: Optional[int] = None, initial_backoff_sec: Optional[float] = None):
        # Dicionário que mapeia o 'peer_id' (string) para o objeto completo 'PeerEntry'.
        # Permite buscas super rápidas de complexidade O(1) pelo identificador do peer.
        self.peers: Dict[str, PeerEntry] = {}

        # Limite máximo de tentativas seguidas de conexão que faremos antes de desistir e marcar como STALE
        self.max_reconnect_attempts = (
            max_reconnect_attempts
            if max_reconnect_attempts is not None
            else Config().max_reconnect_attempts
        )

        # Tempo base (em segundos) que servirá de ponto de partida para o atraso de reconexão exponencial
        # Como o novo sistema de configuração no main não possui

```

```

initial_backoff_sec, usamos 2.0 como padrão.

    self.initial_backoff_sec = (
        initial_backoff_sec
        if initial_backoff_sec is not None
        else 2.0
    )

def update_from_discovery(self, discovery_peers: List[Dict]) -> List
[PeerEntry]:
    """
        Sincroniza a tabela local de peers com a lista de peers
        retornada pelo comando DISCOVER do Rendezvous.

        Como funciona:
        1. Varre a lista de contatos retornada pelo servidor Rendezvous.
        2. Se o peer já existir no seu dicionário local, suas informaçõ
        es de IP/porta/TTL são atualizadas.
        3. Se for um peer novo, uma nova instância de PeerEntry é criada
        , adicionada na tabela e
            colocada em uma lista de retorno (útil para que o cliente
            saiba quem acabou de entrar na rede).

        Retorna:
            - Uma lista contendo as instâncias de novos PeerEntry
            adicionados nessa chamada de sincronização.
    """
    new_peers = []
    for p in discovery_peers:
        name = p["name"]
        namespace = p["namespace"]
        peer_id = f"{name}@{namespace}"
        ip = p["ip"]
        port = p["port"]
        ttl = p.get("ttl", 3600)

        if peer_id in self.peers:
            # O peer já era conhecido, apenas atualizamos seus dados
            de endereço e expiração
            self.peers[peer_id].update_info(ip, port, ttl)

```

```

        else:
            # É um peer novo na rede! Criamos seu registro inicial
na tabela local

            entry = PeerEntry(name, namespace, ip, port, ttl)
            self.peers[peer_id] = entry
            new_peers.append(entry)

    return new_peers

def update_peer(self, peer_id: str, ip: str, port: int, ttl: int):
    """
        Atualiza ou insere um peer na tabela a partir de uma conexão de
entrada (inbound)
        e marca o seu status como "CONNECTED".
    """
    if "@" in peer_id:
        name, namespace = peer_id.split("@", 1)
    else:
        name = peer_id
        namespace = "Default"

    if peer_id in self.peers:
        self.peers[peer_id].update_info(ip, port, ttl)
    else:
        self.peers[peer_id] = PeerEntry(name, namespace, ip, port,
ttl)

    self.mark_connected(peer_id)

def get_peer(self, peer_id: str) -> Optional[PeerEntry]:
    """
        Busca e retorna o objeto completo PeerEntry associado ao peer_id
informado.

        Exemplo:
            peer = peer_table.get_peer("alice@CIC")

        Retorna:
            - 0 objeto PeerEntry contendo todo o estado do peer.

```

```

        - None se o peer_id não estiver cadastrado na tabela.
    """

    return self.peers.get(peer_id)

def mark_connected(self, peer_id: str):
    """
    Marca o peer como conectado com sucesso ("CONNECTED").

    Além de atualizar o status, este método reseta o contador de
    tentativas de reconexão
    e o temporizador de backoff, pois o canal de comunicação agora
    está saudável.
    """
    peer = self.get_peer(peer_id)
    if peer:
        peer.status = "CONNECTED"
        peer.reconnect_attempts = 0
        peer.next_attempt_allowed_at = 0.0

def mark_failed_attempt(self, peer_id: str):
    """
    Registra que uma tentativa de conexão TCP com este peer remoto
    falhou.

    Este método aplica a política de backoff exponencial:
    1. Incrementa o número de tentativas seguidas de falha.
    2. Se o número de falhas atingir ou passar de '
    max_reconnect_attempts', assume que o peer
        está offline definitivamente e muda o status para "STALE" (
    reseta o backoff).
    3. Caso contrário, define o status para "RECONNECTING" e calcula
    a penalidade de tempo:
        atraso = initial_backoff * (2 ^ (tentativas - 1))
    O peer só poderá ser reconectado após esse período de atraso
    ter passado.
    """
    peer = self.get_peer(peer_id)
    if not peer:
        return

```

```

        # Se o peer já estiver ativo e conectado (por exemplo, via uma
        conexão inbound de entrada),
        # não devemos registrar a falha de uma tentativa de conexão
        paralela de saída.

        if peer.status == "CONNECTED":
            return

        peer.reconnect_attempts += 1
        now = time.time()
        peer.last_attempt_time = now

        if peer.reconnect_attempts >= self.max_reconnect_attempts:
            # Excedeu o número limite de tentativas permitidas. O peer é
            marcado como offline.
            peer.status = "STALE"
            peer.next_attempt_allowed_at = 0.0
        else:
            # Aplica o backoff exponencial para evitar sobrecarregar a
            rede com tentativas seguidas de conexão
            # Ex: Tentativa 1 -> 2s, Tentativa 2 -> 4s, Tentativa 3 -> 8
            s, Tentativa 4 -> 16s...
            backoff_delay = self.initial_backoff_sec * (2 ** (peer.
            reconnect_attempts - 1))
            peer.status = "RECONNECTING"
            peer.next_attempt_allowed_at = now + backoff_delay

    def mark_disconnected(self, peer_id: str):
        """
        Muda o status do peer manualmente para "DISCONNECTED".

        Isso acontece quando o encerramento da conexão ocorre de forma
        controlada e amigável
        (por exemplo, quando recebemos ou enviamos uma mensagem 'BYE'
        informando encerramento normal).

        Como não foi uma queda acidental, os contadores de erro e
        backoff são zerados.
        """
        peer = self.get_peer(peer_id)

```

```

    if peer:
        peer.status = "DISCONNECTED"
        peer.reconnect_attempts = 0
        peer.next_attempt_allowed_at = 0.0

def mark_stale(self, peer_id: str):
    """
    Força a marcação de um peer como inativo/offline ("STALE")
    diretamente.

    Útil se o seu programa detectar que o peer foi desregistrado do
    servidor Rendezvous
    ou se você deseja interromper tentativas de reconexão
    imediatamente por algum motivo externo.
    """
    peer = self.get_peer(peer_id)
    if peer:
        peer.status = "STALE"

def remove_peer(self, peer_id: str):
    """
    Remove completamente o registro de um peer da tabela local.
    """
    if peer_id in self.peers:
        del self.peers[peer_id]

def get_all_peers(self) -> List[PeerEntry]:
    """
    Retorna uma lista simples contendo todos os objetos PeerEntry da
    tabela.

    Útil para listar usuários ativos na tela do usuário.
    """
    return list(self.peers.values())

def get_connected_peers(self) -> List[PeerEntry]:
    """
    Retorna uma lista contendo apenas os objetos PeerEntry de peers
    que
    estão atualmente com status "CONNECTED" (conexão TCP ativa).

```

```

    """
    return [p for p in self.peers.values() if p.status == "CONNECTED"
"]

def get_peers_to_reconnect(self) -> List[PeerEntry]:
    """
    Filtra e retorna todos os peers que estão com falha temporária
    ("RECONNECTING")
    e cuja penalidade de tempo de backoff já terminou (ou seja, o
    tempo atual é igual ou maior
    que 'next_attempt_allowed_at').

    Esse método deve ser chamado periodicamente pela thread/tarefa
    de background do cliente
    para saber para quais peers ele deve tentar iniciar um novo
    processo de reconexão.
    """
    now = time.time()
    candidates = []
    for p in self.peers.values():
        if p.status == "RECONNECTING" and now >= p.
next_attempt_allowed_at:
        candidates.append(p)
    return candidates

```


6.10 rendezvous_connection.py

```
# =====
# PROJETO: Chat P2P (Versão Revisada)
# DISCIPLINA: Redes de Computadores
# INSTITUIÇÃO: Universidade de Brasília (UnB) - CIC
# GRUPO: 2
#
# MEMBROS DA EQUIPE:
# - Gabriel Gonçalves Caldo (Matrícula: 231034627)
# - Daniel Rodrigues de Abreu (Matrícula: 241038540)
# - Mario Augusto Vieira dos Santos (Matrícula: 231035778)
#
# ARQUIVO: rendezvous_connection.py
# =====

import asyncio
import logging
from message_router import decode_message
from message_router import encode_message

class RendezvousConnection:
    """
    Classe que representa e gerencia a comunicação com o Servidor
    Rendezvous.

    Seguindo o protocolo especificado, cada interação com o Rendezvous é
    feita
    através de uma conexão TCP curta de comando único (envia uma linha e
    encerra).
    """

    def __init__(self, host, port):
        # Endereço IP/Host do servidor Rendezvous (ex: '45.171.101.167')
        self.host = host

        # Porta TCP do servidor Rendezvous (ex: 8080)
        self.port = port

    async def request(self, msg):
        """
```

```

        Envia uma requisição única para o servidor Rendezvous, aguarda o
retorno,

        fecha a conexão imediatamente e retorna a resposta decodificada.
        """

        # Abre uma nova conexão TCP com o servidor Rendezvous
        reader, writer = await asyncio.open_connection(
            self.host,
            self.port
        )

        # Envia a mensagem codificada (JSON + '\n')
        writer.write(
            encode_message(msg)
        )

        # Garante o envio físico dos dados
        await writer.drain()

        # Lê a única linha de resposta enviada pelo servidor
        data = await reader.readline()

        # Decodifica a resposta JSON para dicionário Python
        response = decode_message(data)

        # Fecha a conexão TCP imediatamente após obter a resposta (
requisito do protocolo)
        writer.close()
        await writer.wait_closed()

        return response

    async def unregister(self, namespace: str, name: str, port: int) ->
dict:
        """
        Envia uma requisição UNREGISTER para o servidor Rendezvous para
remover o cadastro do peer.
        """

        logger = logging.getLogger(__name__)
        unregister_msg = {

```

```

        "type": "UNREGISTER",
        "namespace": namespace,
        "name": name,
        "port": port
    }

    logger.info(f"[Rendezvous] Removendo registro (UNREGISTER) do
peer {name}@{namespace} na porta {port}...")
    try:
        return await self.request(unregister_msg)
    except Exception as e:
        logger.error(f"[Rendezvous] Falha ao enviar requisição
UNREGISTER: {e}")
        return {"status": "ERROR", "message": str(e)}

    async def registration_loop(self, namespace: str, name: str, port:
int, initial_ttl: int):
        """
        Loop de background para re-registrar periodicamente o peer antes
que o TTL expire.
        """
        logger = logging.getLogger(__name__)
        ttl = initial_ttl
        while True:
            try:
                # Intervalo seguro: 80% do TTL atual, garantindo re-
registro bem antes de expirar
                sleep_time = max(5.0, ttl * 0.8)
                logger.debug(f"[Rendezvous] Próxima atualização de
registro em {sleep_time:.1f} segundos (TTL={ttl}).")
                await asyncio.sleep(sleep_time)

                register_msg = {
                    "type": "REGISTER",
                    "namespace": namespace,
                    "name": name,
                    "port": port,
                    "ttl": initial_ttl
                }

                logger.info(f"[Rendezvous] Renovando registro (REGISTER)

```

```

do peer {name}@{namespace} na porta {port}...)

    response = await self.request(register_msg)

    if response.get("status") == "OK":
        ttl = response.get("ttl", initial_ttl)
        logger.debug(f"[Rendezvous] Renovação de registro
bem sucedida. TTL concedido={ttl}.")
    else:
        error_msg = response.get("message", "erro
desconhecido")

        logger.warning(f"[Rendezvous] Falha na renovação de
registro: {error_msg}. Tentando novamente em 10 segundos...")
        # Reduz temporariamente o TTL virtual para tentar
novamente rápido

        ttl = 12.5 # 12.5 * 0.8 = 10s sleep
    except asyncio.CancelledError:
        logger.info("[Rendezvous] Loop de renovação de registro
cancelado.")

        break
    except Exception as e:
        logger.error(f"[Rendezvous] Erro no loop de renovação de
registro: {e}. Tentando novamente em 10 segundos...")

        ttl = 12.5

async def discovery_loop(
    self,
    namespace: str,
    peer_table,
    interval: float,
    exclude_peer_id: str = None,
    active_connections: dict = None,
    local_peer_id: str = None
):
    """
    Loop de background para buscar periodicamente novos peers no
namespace e atualizar a tabela.
    """
    logger = logging.getLogger(__name__)
    while True:

```

```

try:
    discover_msg = {
        "type": "DISCOVER"
    }

    logger.debug("[Rendezvous] Solicitando descoberta (
DISCOVER) global...")
    response = await self.request(discover_msg)

    if response.get("status") == "OK":
        peers = response.get("peers", [])
        if exclude_peer_id:
            peers = [p for p in peers if f"{p.get('name')}@{
p.get('namespace')}" != exclude_peer_id]
            logger.debug(f"[Rendezvous] Descoberta bem sucedida.
Encontrado(s) {len(peers)} peer(s) registrado(s).")
            new_peers = peer_table.update_from_discovery(peers)
            if new_peers:
                for np in new_peers:
                    logger.info(f"[Rendezvous] Novo peer
descoberto: {np.peer_id} em {np.ip}:{np.port}")

                    # Conectar automaticamente a todos os peers
descobertos que não estão conectados
                    if active_connections is not None and local_peer_id
is not None:
                        for p_info in peers:
                            p_id = f"{p_info['name']}@{p_info['namespace
'],'}"

                            if p_id == exclude_peer_id:
                                continue

                            peer_entry = peer_table.get_peer(p_id)
                            if peer_entry and peer_entry.status == "
DISCONNECTED" and p_id not in active_connections:
                                asyncio.create_task(
                                    self._connect_to_peer(peer_entry,
local_peer_id, peer_table, active_connections)
                                )

        else:

```

```

        error_msg = response.get("message", "erro
desconhecido")

        logger.warning(f"[Rendezvous] Falha na descoberta: {
error_msg}")

    except asyncio.CancelledError:
        logger.info("[Rendezvous] Loop de descoberta (DISCOVER)
cancelado.")

        break

    except Exception as e:
        logger.error(f"[Rendezvous] Erro no loop de descoberta:
{e}")

    try:
        await asyncio.sleep(interval)
    except asyncio.CancelledError:
        logger.info("[Rendezvous] Loop de descoberta cancelado
durante a pausa.")

        break

async def _connect_to_peer(self, peer, local_peer_id, peer_table,
active_connections):
    """
    Inicia uma conexão TCP em background com o peer informado.
    """

    logger = logging.getLogger(__name__)

    # Verifica novamente o status para evitar race conditions
    if peer.status == "CONNECTED" or peer.peer_id in
active_connections:
        return

    # Define um placeholder temporário para atuar como trava
    active_connections[peer.peer_id] = "connecting"

    from peer_connection import PeerConnection
    conn = PeerConnection(
        local_peer_id,
        peer_table,
        on_close=lambda c: active_connections.pop(c.remote_peer_id,
None) if c.remote_peer_id else None

```

```
)  
  
try:  
    logger.info(f"[Descoberta] Iniciando conexão automática com  
peer descoberto {peer.peer_id} em {peer.ip}:{peer.port}...")  
    await conn.connect(peer.ip, peer.port)  
    # Inicia escuta da conexão  
    asyncio.create_task(conn.listen())  
    active_connections[peer.peer_id] = conn  
    logger.info(f"[Descoberta] Conexão automática com {peer.  
peer_id} estabelecida com sucesso")  
except Exception as e:  
    err_msg = f"{type(e).__name__}: {e}" if str(e) else type(e).  
__name__  
    logger.warning(f"[Descoberta] Falha na conexão automática  
com {peer.peer_id}: {err_msg}")  
    active_connections.pop(peer.peer_id, None)  
    peer_table.mark_failed_attempt(peer.peer_id)
```

6.11 reconnection_manager.py

```
# =====
# PROJETO: Chat P2P (Versão Revisada)
# DISCIPLINA: Redes de Computadores
# INSTITUIÇÃO: Universidade de Brasília (UnB) - CIC
# GRUPO: 2
#
# MEMBROS DA EQUIPE:
# - Gabriel Gonçalves Caldo (Matrícula: 231034627)
# - Daniel Rodrigues de Abreu (Matrícula: 241038540)
# - Mario Augusto Vieira dos Santos (Matrícula: 231035778)
#
# ARQUIVO: reconnection_manager.py
# =====

import asyncio
import logging
import time
from peer_connection import PeerConnection
from peer_table import PeerTable
from typing import Dict

async def _attempt_reconnect(peer_id, peer, peer_table,
    outbound_connections, logger):
    if peer.status == "CONNECTED" or peer.peer_id in
    outbound_connections:
        logger.debug(f"[Reconnect] Conexão ativa ou em andamento já
        existe para {peer.peer_id}. Cancelando reconexão.")
        return
    logger.info(f"[Reconnect] Tentando reconectar automaticamente a {
    peer.peer_id} em {peer.ip}:{peer.port}")
    conn = PeerConnection(
        peer_id,
        peer_table,
        on_close=lambda c: outbound_connections.pop(c.remote_peer_id,
        None) if c.remote_peer_id else None
    )
    try:
```



```

        await conn.connect(peer.ip, peer.port)
        listen_task = asyncio.create_task(conn.listen())

        # Se a identidade do peer mudou (ex: reaproveitamento de IP)
        if conn.remote_peer_id and conn.remote_peer_id != peer.peer_id:
            peer_table.mark_stale(peer.peer_id)
            outbound_connections[conn.remote_peer_id] = conn
            logger.info(f"[Reconnect] Peer mudou de identidade. Era {
peer.peer_id}, agora é {conn.remote_peer_id}")
        else:
            outbound_connections[peer.peer_id] = conn
            logger.info(f"[Reconnect] Reconectado com sucesso a {peer.
peer_id}")

    except Exception as e:
        err_msg = f"{type(e).__name__}: {e}" if str(e) else type(e).
__name__
        logger.warning(f"[Reconnect] Falha ao reconectar a {peer.peer_id
}: {err_msg}")
        peer_table.mark_failed_attempt(peer.peer_id)

async def reconnection_loop(
    peer_id: str,
    peer_table: PeerTable,
    outbound_connections: Dict[str, PeerConnection],
    interval: int = 5
):
    """
    Loop em segundo plano que verifica periodicamente se existem peers
    no estado
    'RECONNECTING' cujo tempo de penalidade (backoff) já expirou, e então
    tenta
    estabelecer uma nova conexão TCP de forma autônoma.
    """
    logger = logging.getLogger(__name__)
    try:
        while True:
            await asyncio.sleep(interval)
            candidates = peer_table.get_peers_to_reconnect()

```

```
        # Lança tarefas concorrentes para não bloquear o loop caso
uma das conexões sofra timeout
        for peer in candidates:
            # Previne múltiplas tentativas simultâneas para o mesmo
peer jogando o timer para o futuro
            peer.next_attempt_allowed_at = time.time() + 60.0
            asyncio.create_task(_attempt_reconnect(peer_id, peer,
peer_table, outbound_connections, logger))

except asyncio.CancelledError:
    pass
```